OVERVIEW OF COMPUTER PROGRAMS AND THE JAVA PROGRAMMING LANGUAGE

CONTENTS OF THE CHAPTER

- *Basic concepts of computers and programs.*
- *How a program works on a computer.*
- *Introduction to the Java programming language.*
- *Features of Java.*
- *Java Virtual Machine (JVM).*
- *Setting up the Java application development environment.*

Chapter 1 introduces basic concepts of computers and computer programs in general, and Java programs in particular. The content of this chapter provides an overview of how an electronic computer runs a program. This knowledge will be the foundation for developing application software using any programming language. This chapter also focuses on introducing the Java programming language, including its development history, the Java virtual machine, the features of this programming language, and instructions on how to set up an environment for learning, researching, and developing applications.

1.1. COMPUTERS AND PROGRAMS

1.1.1. Basic Concepts of Computers

A computer is an electronic device composed of many components. The **central processing unit** [1] **(CPU) is the brain of the** computer, where the computing tasks are performed. The CPU's function is to execute the code of programs. At the machine language level, the CPU's operation is quite simple. The CPU executes a program stored as a sequence of machine

[1] Also known as the CPU, it is the component in a computer responsible for executing computer program instructions by performing arithmetic, logic, comparison, and data input/output operations.

language instructions[2]**.** In **main memory**[3] **(RAM)**, it performs this by continuously reading or fetching an instruction from memory and then executing that instruction. The process of fetching and executing an instruction is called a cycle. A CPU operates quickly by performing many cycles in a unit of time. Typically, a CPU executes one instruction at a time. However, most modern CPUs contain multiple cores, allowing them to execute several instructions simultaneously. The CPU's ability to read instruction codes from main memory is accelerated by cache **memory**[4].

A CPU is made up of millions of transistors arranged on a small circuit board. The core of a CPU is usually divided into two main blocks called the **control unit (**CU**)** and the **arithmetic logic unit (**ALU**)**. Below is an image of an Intel Core i9 CPU:

Figure 1. Image of an Intel Core i9 CPU.

1.1.2. Computer Programs

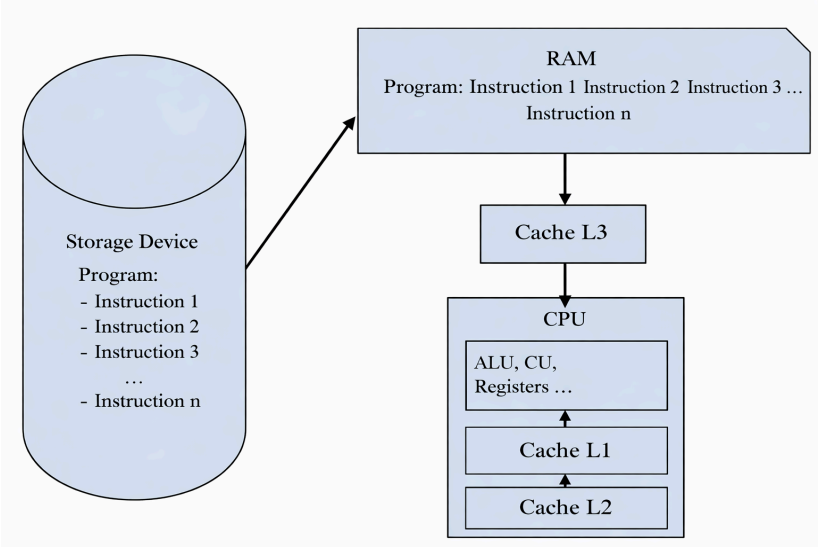
A **program** is a sequence of instructions **that** a computer will execute

[2] Also known as machine code, it is the set of instructions that the computer's CPU executes directly.

[3] It's where all the programs/applications currently running on the computer are stored.

[4]This area temporarily stores instructions or data that the CPU needs to process.

to perform a specific task requested by the user. A computer can only understand and execute instructions written in a language called **machine language**. Therefore, program instructions written in any **programming language** are compiled into machine language, stored on **a storage device**, and executed by **the CPU.**



When a computer executes a program, that program is loaded from **storage devices** into the computer's **main memory. Besides the instructions** to execute the program, memory can also contain **data** being used or processed by the program. Main memory consists of addressable memory cells. An **address** provides a location to retrieve a specific piece of information from among the many pieces of information stored in memory. When the CPU needs to access a program instruction or data at a specific location, it sends the address of that information as a signal to memory; memory responds by sending back the value contained in the specified location. The CPU can also store information in memory by specifying what information should be stored and the address of the location where it will be stored. The CPU's access to memory to read instructions and data is often supported by caches to speed up access. However, it should be noted that, not all data is stored and passed through the levels of the cache (such as L1, L2, L3) because these memory areas have very limited capacities compared to main memory and are usually only used to store frequently accessed

instructions or data.

1.2. JAVA PROGRAMMING LANGUAGE

1.2.1. History of the Java programming language

Java is an object-oriented, class-based programming language designed to have as few dependencies during execution as possible. It is a write-once, run-anywhere (**WORA**) programming language, meaning that compiled Java code can run on all Java-supported platforms without needing to be recompiled. Java applications are typically compiled into **bytecode**. This bytecode can run on any **Java Virtual Machine** (**JVM**) regardless of the underlying computer architecture. Java's syntax is similar to C and C++. According to GitHub, Java is currently one of the most popular programming languages used by programmers.

Java was originally developed by **James Gosling** at **Sun Microsystems** (later acquired by Oracle) and released in 1995 as a core component of Sun Microsystems' Java platform. The Java compilers, virtual machine, and original and reference executable class libraries were initially released by Sun under a proprietary license. Since May 2007, in accordance with the Java Community Process specifications, Sun has licensed most of its Java technologies under the GNU General Public License (GPL). After being acquired by Oracle, Java was developed in parallel with the **OpenJDK**, which is free, open-source software, and the **J**ava **V**irtual **M**achine (**JVM**) is distributed across almost all platforms.

Some key milestones in Java’s development include:

- **In 1990**, James Gosling, working at Sun Microsystems, created the **Oak** language, named after a type of tree outside Gosling's office.
- **In 1995**, Oak was renamed **Java** (a type of Indonesian coffee).
- **In 2009**, Oracle acquired Sun Microsystems, continuing to maintain and develop Java.

The name Java and the coffee cup icon were chosen for this

programming language after the development team searched for a new name for the Oak language. Both the name and the icon for Java satisfied the team's criteria because it was revolutionary, vibrant, familiar, and easy to remember. The Java icon was maintained for many years, as illustrated below:

Figure 1.3. Icons of the Java programming language

Java is a programming language classified as a **high-level programming language**, along with modern languages such as C++, C#, Python, JavaScript, etc. The classification of programming languages is specifically shown as follows:

- High-level language: **C++, Java, C#, Python, etc.**
- Middle-level language**: C**
- Low-level language**: Assembly language.**
- Machine language: **consists of bits 1 and 0.**

1.2.2. Applications of Java

Java is used in a wide variety of real-world applications, from e-commerce websites to mobile apps, from scientific applications to financial applications such as e-trading systems, from games like Minecraft to big data processing platforms.

If you want to see the applications of Java, consider Android applications; most of them are written in the Java programming language, using **Google's Android SDK** (similar to **JDK**). In recent years, Android

has developed rapidly, and many Java programmers have become Android application developers. Java plays a significant role in **financial services**. Many banks and financial corporations such as Goldman Sachs, Citigroup, Barclays, Standard Chartered, etc., use it. Java is used to write transaction systems, verification and auditing systems, data processing projects, and several other critical tasks. Java is primarily used to write server-side applications; it receives data from one computer, processes it, or sends data to another computer for processing. Java is also widely used in web and e-commerce applications. Many web applications are created using Spring MVC, Struts 2.0, and similar frameworks. Even simple web applications based on Servlets or JSP are very popular. Many government, healthcare, insurance, education, and defense organizations have web applications built using Java.

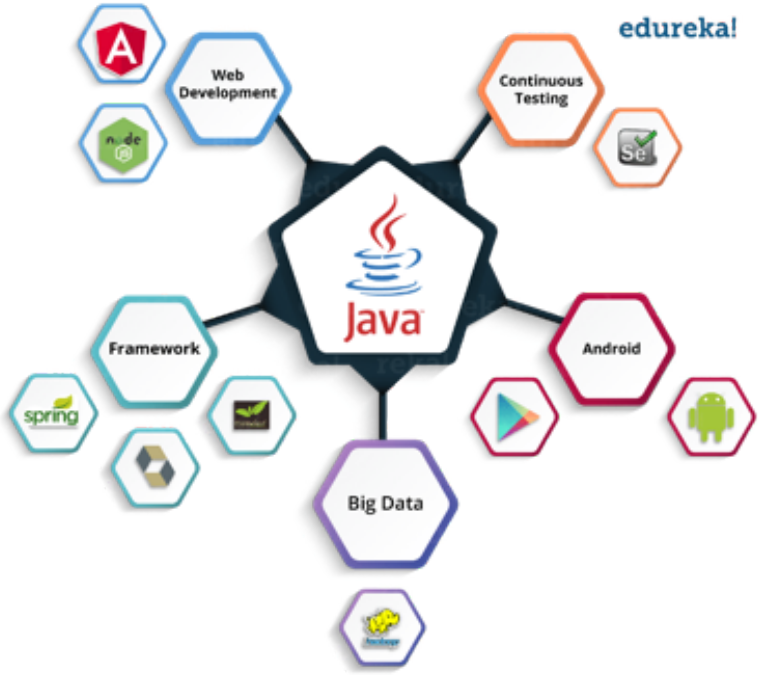


Figure 1.4. Applications of Java in various fields.

Many useful **development tools** and **software** are written in Java, for example, Eclipse, IntelliJ IDEA, and Netbeans IDE. For developing computer applications with graphical interfaces, the Swing library is very

popular and widely used. However, in recent times, JavaFX has become more developed and is gradually replacing Swing.

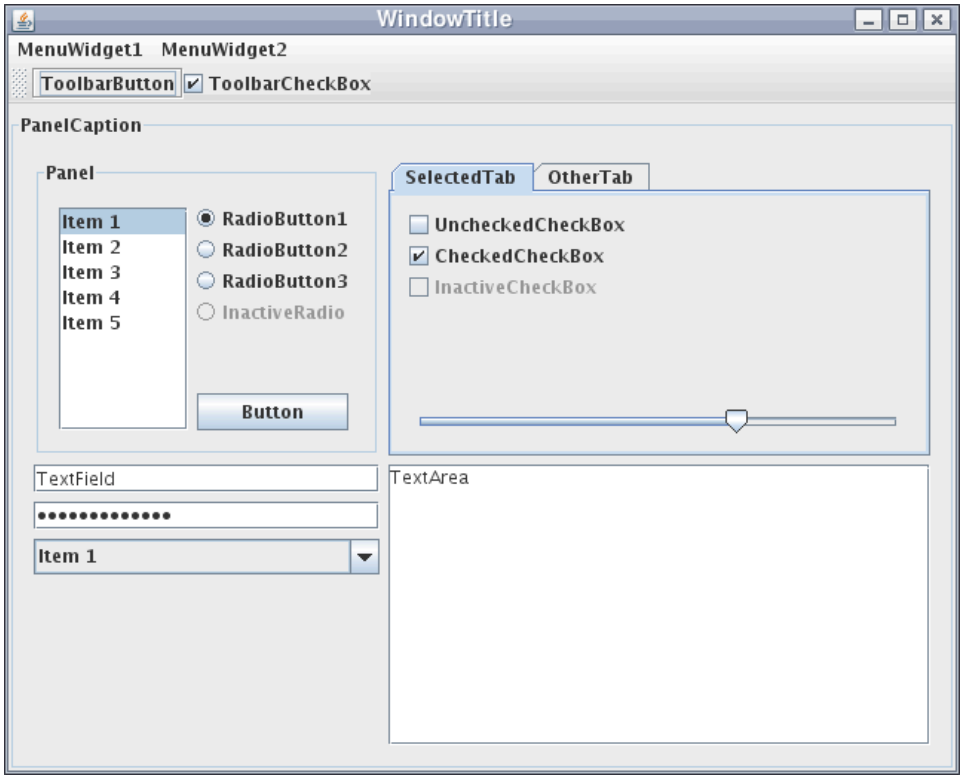


Figure 1.5. Java application interface created by the Swing library.

Java is also widely used in embedded applications, demonstrating the platform's broad applicability. A device only needs 130KB to utilize Java technology (such as on a smart card or sensor). This also reflects the "write once, run everywhere" philosophy of Java developers. Java is also used in **big data processing technologies**. **Hadoop** and other big data technologies are using Java; for example, HBase, Accumulo (open source), and Apache's **ElasticSearch** are Java-based. While Java is not dominant in this field, and technologies like MongoDB are written in C++, Java has the potential to gain a larger market share thanks to the popularity of Hadoop and ElasticSearch.

Java's performance has been greatly improved with the **JIT (Java-In-Time) compiler**, achieving performance levels comparable to

C++. Therefore, Java is also widely used for writing high-performance systems. While performance may be slightly inferior to C or C++, it offers added security, flexibility, and ease of maintenance. Currently, Java is also frequently used in science. The main reason for this is that Java is secure, portable, easy to maintain, and comes with better tools than C++ or any other language.

1.2.3. Java Versions

As of September 2025, the latest version is **Java 25**. Oracle recommends that users use the **LTS version.** Java LTS (Long Term Supported) is for application development, while other versions should only be used for testing. As the name suggests (Long Term Supported), LTS versions are those with long-term support. The most recent Java LTS version is Java **25** , released in September 2025. In addition, Oracle has also released updates for **Java 8, 11, 17, 21 LTS**. Oracle advises users to uninstall outdated Java versions due to serious risks stemming from unresolved security issues. A detailed list of Java versions is shown in the table below:

Table 1-1. List of Java versions

Version	Release date
JDK Beta	1995
JDK 1.0	January 1996
JDK 1.1	February 1997
J2SE 1.2	December 4, 1998
J2SE 1.3	May 8, 2000
J2SE 1.4	February 13, 2002
Java SE 5	September 29, 2004
Java SE 6	December 11, 2006
Java SE 7	July 28, 2011
Java SE 8 (LTS)	March 18, 2014
Java SE 9	September 21, 2017

Version	Release date
Java SE 10	March 20, 2018
Java SE 11 (LTS)	September 25, 2018
Java SE 12	March 19, 2019
Java SE 13	September 17, 2019
Java SE 14	March 17, 2020
Java SE 15	September 16, 2020
Java SE 16	March 16, 2021
Java SE 17 (LTS)	September 14, 2021
Java SE 18	March 22, 2022
Java SE 19	September 20, 2022
Java SE 20	March 21, 2023
Java SE 21 (LTS)	September 19, 2023
Java SE 22	March 19, 2024
Java SE 23	September 17, 2024
Java SE 24	March 18, 2025
Java SE 25 (LTS)	September 16, 2025

It can be seen that Oracle releases Java versions quite regularly, usually in **March** and **September** each year.

1.2.4. Features of Java

Compared to earlier languages, Java is considered **simple and compact**. Learning and using Java is also much easier. Specifically, Java doesn't use header files; it has no pointers; no operator overloads; no virtual base classes; no memory allocation management; and no goto statements. However, Java is also very accessible because it uses a syntax similar to C and C++, so learners don't need to spend too much time getting familiar with it. Besides, Java is very compact. The basic Java compiler is only 40KB, while the standard library is 175KB. To solve the same problem, Java uses a shorter, simpler syntax than C and C++. Java is more suitable for

building large systems, something that earlier languages struggled with and required more effort to develop.

Java is an Object-oriented **programming** language. When programming in Java, everything except for basic data types is an object. Java does not use global methods; methods must be called through objects/classes. Typical features of object-oriented programming, including inheritance, polymorphism, encapsulation, and abstraction, are easily implemented in Java. However, Java does not support multiple inheritance.

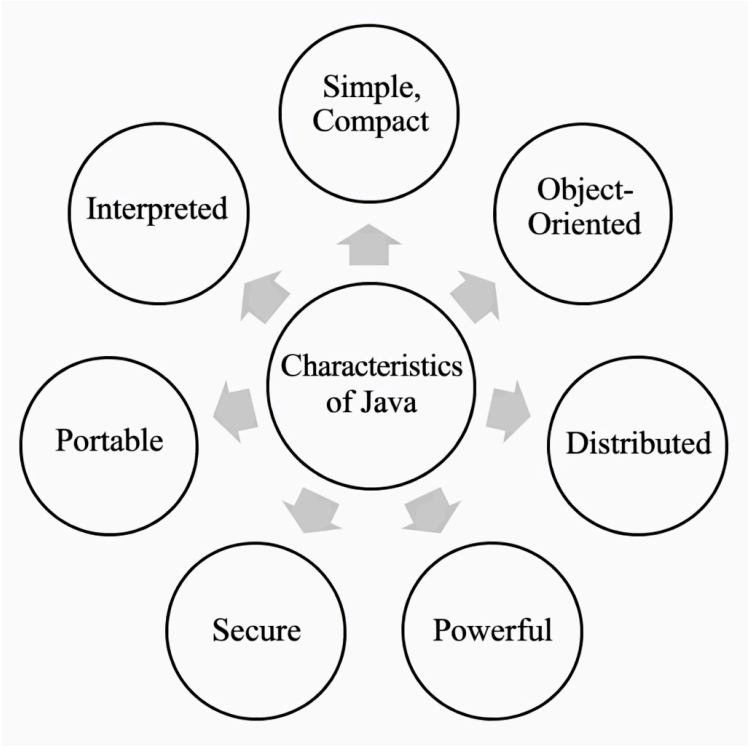


Figure 1.6. Characteristics of Java

Distributed is demonstrated by its support for building network applications, such as libraries for working with network protocols, input/output libraries for data transmission and reception, and mechanisms that allow working with distributed data similarly to local data. Today, distributedness is a standard requirement; however, since 1995, most programming languages have not supported this. Java is also very powerful in developing web applications.

Powerful programming language because it allows working with many different data types, has an optimized memory management mechanism, and automatically cleans up and reclaims unused objects. It is also one of the programming languages capable of exception handling, helping to detect errors during execution. Its robustness is also demonstrated by its ability to create distributed, multi-threaded applications. Furthermore, Java allows the execution of source code written by lower-level languages using the `**native**` keyword .

The Java programming language also has **high security**. It does not allow the use of pointers, so it is very safe because it limits direct access to memory. Java applications run in a controlled environment (the Java virtual machine). Security is also demonstrated by restricting access to data/methods and objects through access modifiers.

Java 's portability is demonstrated by the fact that programs can be written and run anywhere (WORA – Write Once, Run Anywhere). To achieve this, Java uses bytecode, which are code snippets that can be executed on virtual machines regardless of the platform. To achieve high portability, the size of the virtual machine is very compact. Java applications rarely require installation and can be run on many platforms using different architectures or operating systems. For example, Windows applications (exe) have executable files for both 32-bit and 64-bit environments, while Java applications have a single version.

Interpretation is also an important feature of this programming language. It is achieved by compiling the original source code into bytecode and then interpreting this bytecode. Java also has the JShell engine, which allows for immediate execution and display of command results.

1.2.5. Comparing Java with C and C++

Java is often compared to earlier languages like **C** and **C++** , and although Java's syntax is inherited from C and C++, significant differences still exist between these languages. These differences are shown in the table below:

Table 1.2. Comparison between Java and two other languages, C and C++:

	C	C++	Java
Programming methods	Procedural	Object-oriented	Object-oriented
Developed from	Assembly language	C language	C and C++ languages
Created by	Dennis Ritchie, 1972	Bjarne Stroustrup, 1979	James Gosling, 1991
Compile	Compile	Compile	Compile + Interpret
Platform dependence	Platform dependent	Platform dependent	Platform Independent
Execute code	Direct	Direct	Through the Java Virtual Machine
Approach	Top -down	Bottom-up	Bottom -up
The generated file	.exe file (Windows)	.exe file (Windows)	.class file
Preprocessing instructions	(#include, #define)	(#header, #define)	import
Number of keywords	32 keywords	63 keywords	50 keywords
Type of structure/set	Support	Support	Not supported
Multiple inheritance	No OOP	There is multiple inheritance.	There is no multiple inheritance.
Overloading	Do not have	Operator and Method overloading	Method overloading
Pointer	Have	Have	Do not have
Memory allocation	Use malloc, calloc	Use new and delete.	Automatically via Garbage Collector

	C	C++	Java
Exception handling	Not supported	Support	Support
Generic	Not supported	Support	Support
Constructor/Destructor	There is no create/destroy method.	Have	There is no destroy method.
Multiple Thread	New	New	It's been there since the beginning.
Storage class	Yes	Yes	Do not have

1.3. JAVA VIRTUAL MACHINE

1.3.1. Java Virtual Machine

In other programming languages, **the compiler** compiles **the original source code** written in that programming language into **code for a specific system**. This is code that the system can read and understand. The system then converts this code into **machine code** and executes it. However, the **Java compiler** creates code for a virtual machine called **the Java Virtual Machine** (JVM). This code is called **bytecode**. This bytecode is executed on the Java virtual machine through an **interpretation process. Java virtual machines are developed for different systems, so application programmers do** n't need to worry about the platform **on which** the program will be executed. The Java virtual machine is always packaged within the Java Runtime **Environment** (**JRE**). Therefore, the Java virtual machine is:

- This is where Java bytecode is converted into a machine-understandable language.
- Part of JRE.
- The environment for running Java applications.

The way a program is compiled and executed is illustrated in the figure below and will be discussed in more detail later. Figure 1-7 shows

Java virtual machines (**JVMs**) installed on different platforms such as Win32, Unix, and MacOS. Java source code is compiled and executed on these platforms.

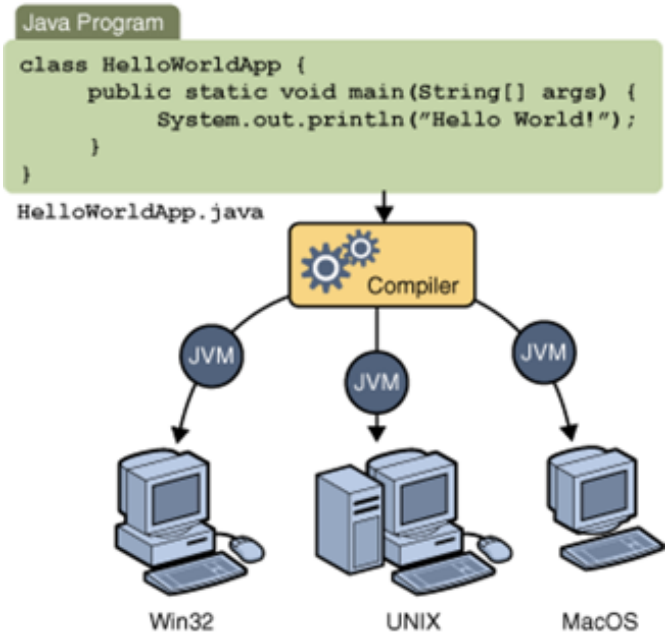


Figure 1.7 -. How a Java program can run on multiple platforms.

1.3.2. Compiling and executing programs written in C

To understand the compilation process in Java, let's take a quick look at the **compiling** and **linking processes** in C. These are two crucial processes for making a C-based program work on a computer. First, to write and execute a software program, the following components are needed:

- **Editor** - A text editor, used for writing source code.
- **Compiler** - A tool used to convert source code into machine code.
- **Linker** - A linker, used to combine different program files within the main program.
- **Loader** - A loader used to load executable files into memory (RAM).

Consider a simple and typical C program example. It contains three

files: a1.c, a2.c, and a3.c:

- **main** function is stored in the file a1.c.
- The function **f1** is saved in the file a2.c.
- The function **f2** is saved in the file a3.c.

All these files, namely **a1.c** , **a2.c** , and **a3.c** , are fed into **the compiler .** The compiler will create the corresponding object sets **a1.obj** , **a2.obj** , and **a3.obj** . The next step is **the linker.** (**Linker**) will integrate all these object files into one **. An executable file (a file with the .exe extension)** . During program execution, a loader **will** load the executable file **a.exe into RAM** for execution.

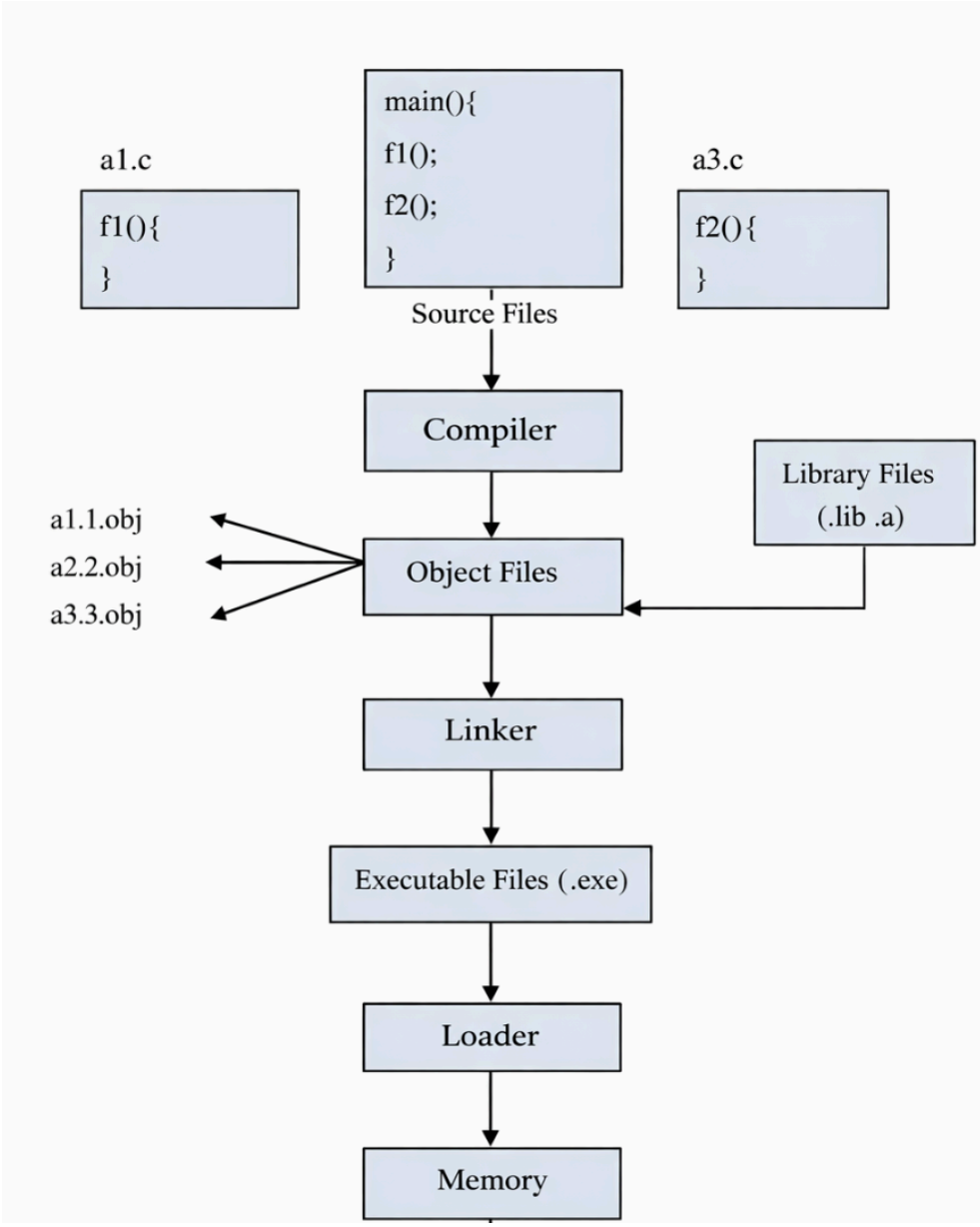


Figure 18. -The process of compiling and executing a program written in C.

1.3.3. Compiling and executing programs written in Java

Let's consider this process for **JAVA** . Consider a similar program where there is a main method and two other methods f1 and f2 stored in files:

- **main** method is stored in the a1.java file.
- Method **f1** is stored in the file as a2.java.
- Method **f2** is stored in the file as a3.java.

If you look closely, you'll see that in Java, the term commonly used is **"method"** instead of **"function** ." This is because **methods** are typically used in object-oriented programming, while **functions** are used in procedural programming. All three methods in the three source code files are used in the program. **The compiler** compiles the three files and creates three corresponding .class files containing bytecode. Unlike C, Java does not have a linker. Java uses a class loader to load these classes into main memory (**RAM). The** Java virtual **machine is** already **running** in **main** memory. This virtual machine contains **an interpreter** that converts the bytecode into machine code. The following describes the Java code compilation process in the JVM:

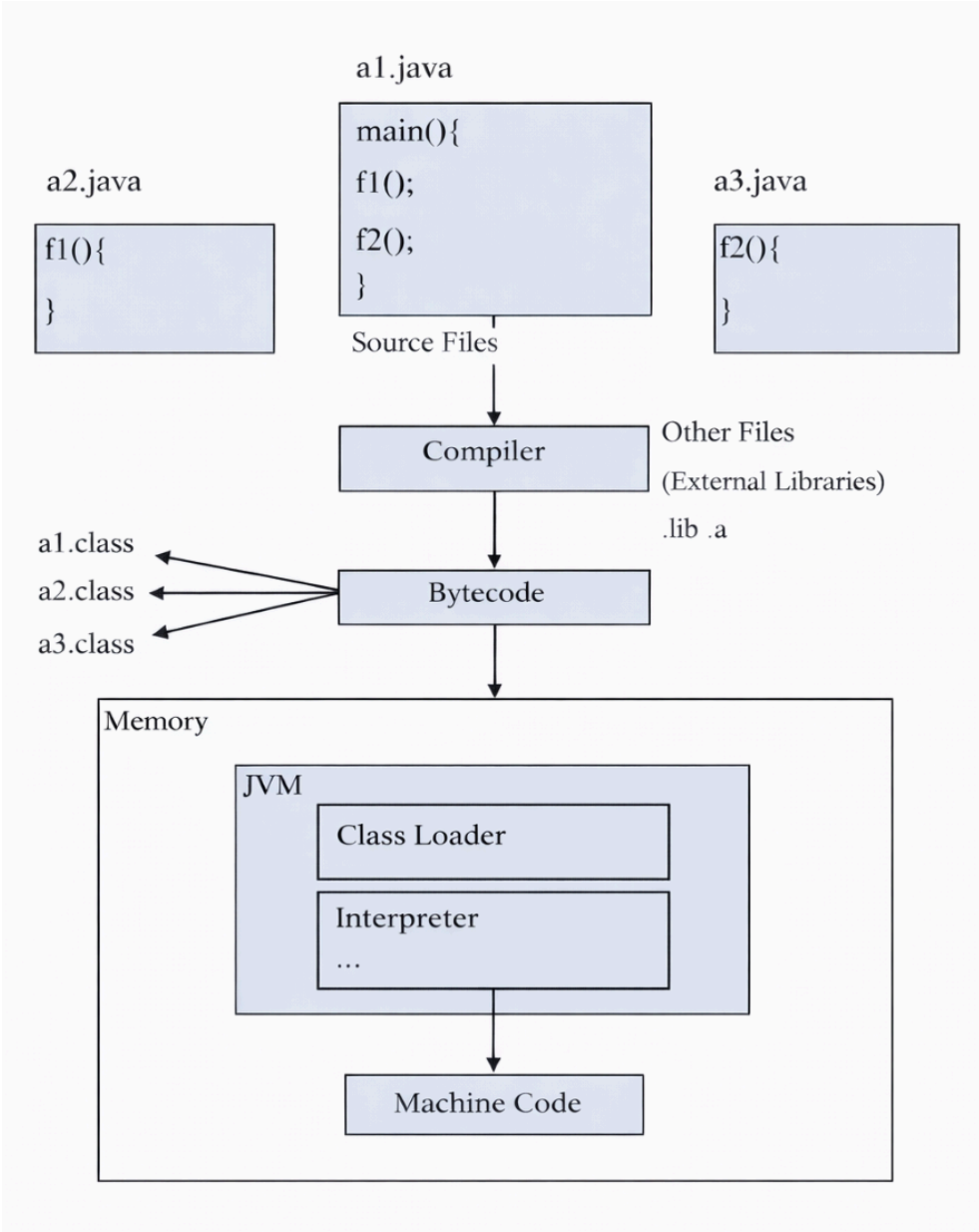


Figure 1.9. The process of compiling and executing a program written in Java.

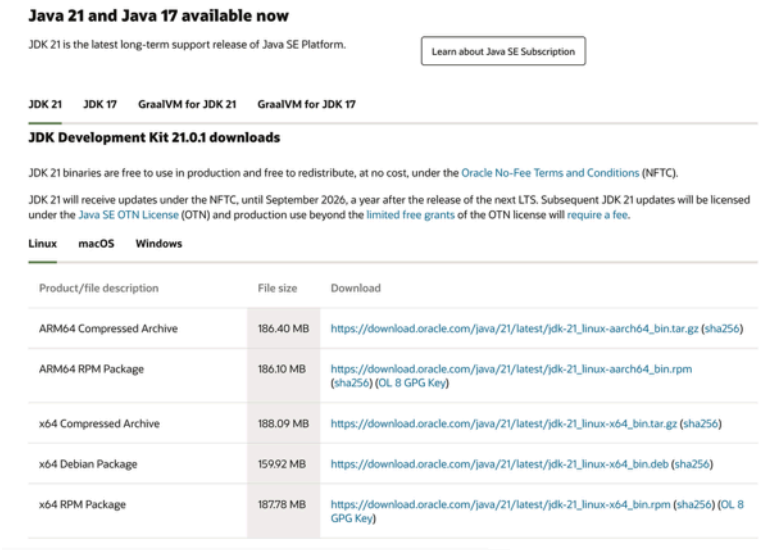
Thus, it can be seen that for a Java program to run, both a compiler and an interpreter are needed. **A compiler** is a program that converts a program from one language level to another. The Java compiler converts high-level Java code into bytecode (which is also a type of machine code). An **interpreter** is a program that executes bytecode line by line after it has

been compiled. Therefore, it can be said that **a Java program is both compiled and interpreted. Files with the .java** extension contain class definitions, which are then compiled into ***.class files** containing bytecode. These class files can be packaged into a JAR file (Java Archives). When JAR files are placed in library repositories, these JAR files are also called Artifacts. A desktop application is usually packaged as a JAR file, while a web project is typically packaged as a WAR (Web Archive) file.

1.4. SETTING UP THE JAVA ENVIRONMENT

1.4.1. Download the Java development kit

If you want to set up an environment for using the Java language, go to: **https://www.oracle.com/java/technologies/downloads.** There, download the **Java Development Kit** (**JDK**). Note that the downloaded version must be compatible with your **computer 's** platform **.**



Java 21 and Java 17 available now

JDK 21 is the latest long-term support release of Java SE Platform.

Learn about Java SE Subscription

JDK 21 JDK 17 GraalVM for JDK 21 GraalVM for JDK 17

JDK Development Kit 21.0.1 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions (NFTC).

JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the Java SE OTN License (OTN) and production use beyond the limited free grants of the OTN license will require a fee.

Linux macOS Windows

Product/file description	File size	Download
ARM64 Compressed Archive	186.40 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-aarch64_bin.tar.gz (sha256)
ARM64 RPM Package	186.10 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-aarch64_bin.rpm (sha256) (OL 8 GPG Key)
x64 Compressed Archive	188.09 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-x64_bin.tar.gz (sha256)
x64 Debian Package	159.92 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-x64_bin.deb (sha256)
x64 RPM Package	187.78 MB	https://download.oracle.com/java/21/latest/jdk-21_linux-x64_bin.rpm (sha256) (OL 8 GPG Key)

Figure 1 -10. Download Java versions for different platforms

The downloaded file may be **a compressed file** (zip file) or **an installation file** (exe, msi file). Installation will be easy and quick with the installation files. Then, just follow a few simple steps to successfully install the Java application development environment.

1.4.2 . Installing and testing Java

After downloading **Java** , to install it, select the **installation directory** and then click the next steps to install Java on your computer. Once the JDK is installed, it will be installed on your computer and the necessary configurations will be automatically set up.

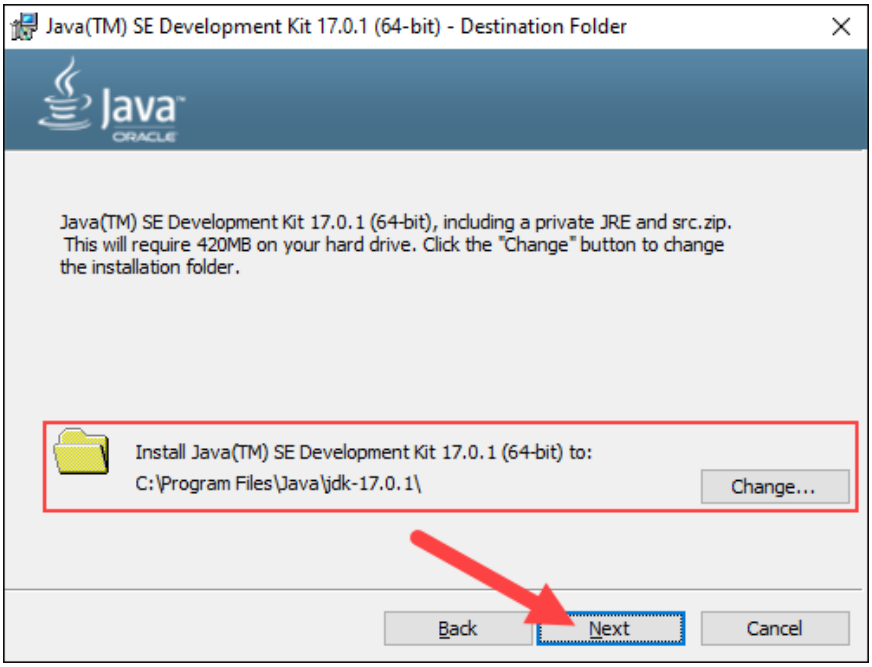


Figure 1.11 -. Java installation process on a Windows computer .

To check if Java is installed on your machine and if the path is successfully set, use the following command in **the command line window** (e.g., **Command Prompt** on Windows or **Terminal** on Mac):

```
java -version
```

The results will appear to be something like this:

```
    java version "17.0.4" 2022-07-19 LTS
    Java(TM)    SE    Runtime    Environment    (build
17.0.4+11-LTS-179)
    Java    HotSpot(TM)    64-Bit    Server    VM    (build
17.0.4+11-LTS-179, mixed mode, sharing)
```

To check if the Java Compiler is installed on your machine and if the path is successfully set, use the command:

```
javac -version
```

Result:

```
javac 17.0.4
```

To write your first Java program, use any **code editor to create a file named HelloWorld.java** with the following content:

```
public class HelloWorld {
   public static void main ( String [] args) {
       System . out .println( "Hello Java" );
   }
}
```

Compile the above program by running the following command:

```
java HelloWorld.java
```

The above command will create the file **HelloWorld.class** . Execute the bytecode in this file by running:

```
Java HelloWorld
```

The results are displayed:

```
Hello Java
```

Thus, we have successfully set up the compilation and execution environment for the Java program.

1.5 . TOOLS FOR PRACTICAL USE AND APPLICATION DEVELOPMENT IN JAVA

The JDK downloaded from Oracle's website already includes **a compilation and execution environment** for Java code. However, to write Java programs more easily, a separate source code editor or a **more functional application development toolkit is needed** . Currently, there are many types of these development tools, including **professional application development tools** and **online Java coding and programming**

practice tools . Both types of tools are suitable for practicing basic Java knowledge.

1.5.1. Online Java Practice Tools

Several **online tools** allow Java programming directly in the browser. These tools often have limited features, but they enable programmers, primarily **learners,** to practice short and simple Java code snippets. Many online practice tools exist, such as:

1. https://www.tutorialspoint.com/compile_java_online.php
2. https://www.jdoodle.com/online-java-compiler
3. https://www.programiz.com/java-programming/online-compiler
4. https://onecompiler.com/java
5. https://www.w3schools.com/java/tryjava.asp
6. https://www.online-java.com
7. https://fit.neu.edu.vn/codelab

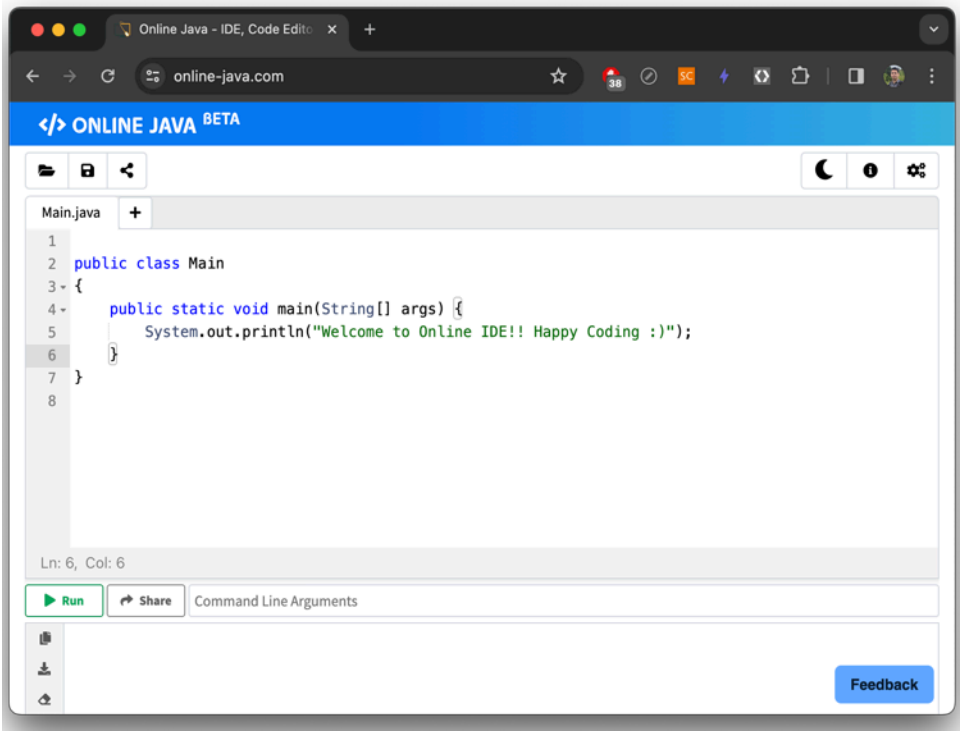


Figure 1.12. Online Java programming tools

1.5.2 . Professional Application Development Tools

Popular professional application development tools for Java today include: **Netbeans**, **Eclipse**, and **IntelliJ IDEA.**

NetBeans is a free, open-source integrated development environment (IDE). It works well on many different operating systems such as Linux, Windows, macOS, etc., providing the most essential features for developing applications across various platforms. NetBeans is widely used for developing Java applications; however, the software is quite large and usually requires machines with relatively high RAM and CPU to run. Besides Java, NetBeans supports many other programming languages such as Python, Ruby, JavaScript, Groovy, C/C++, and PHP. The software interface is also very user-friendly.

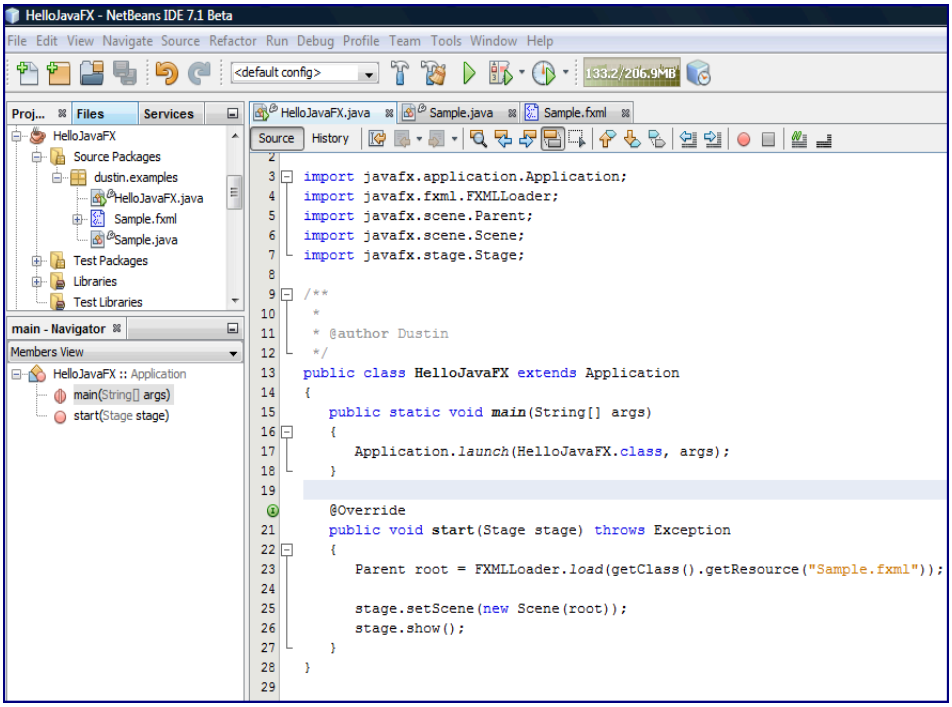


Figure 1.13. Main interface of the Netbeans tool.

Eclipse is an integrated development environment (IDE) developed by the open-source community and IBM. Eclipse is an integrated Java development environment. With Eclipse, programmers can extend

functionality by adding plugins.

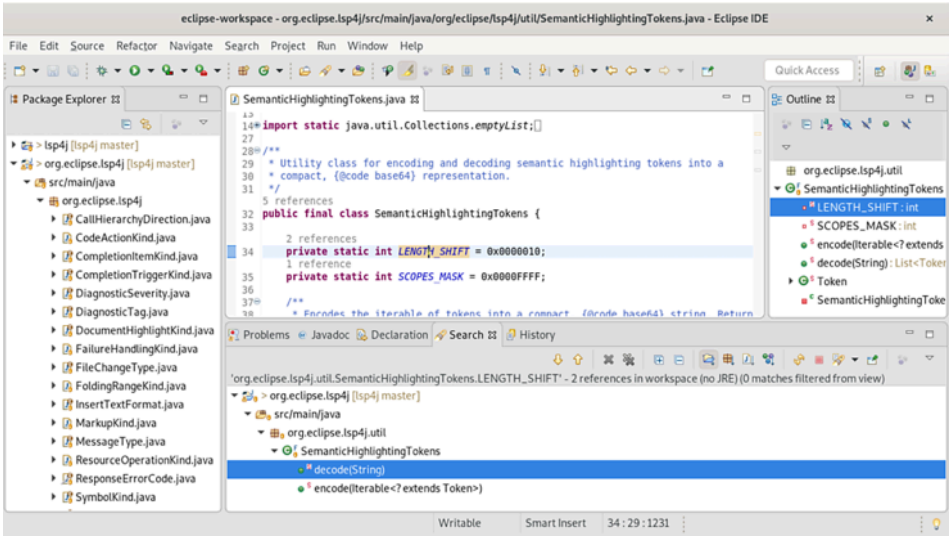


Figure 114. -Main interface of the Eclipse programming tool.

IntelliJ IDEA is also a popular Java development tool developed by JetBrains. It's available in both **community** and **ultimate versions**. Programmers can choose either the **Ultimate** or **Community version** to learn basic Java. Using the **Ultimate version** requires a fee or login with **an .edu account** to activate and use it.

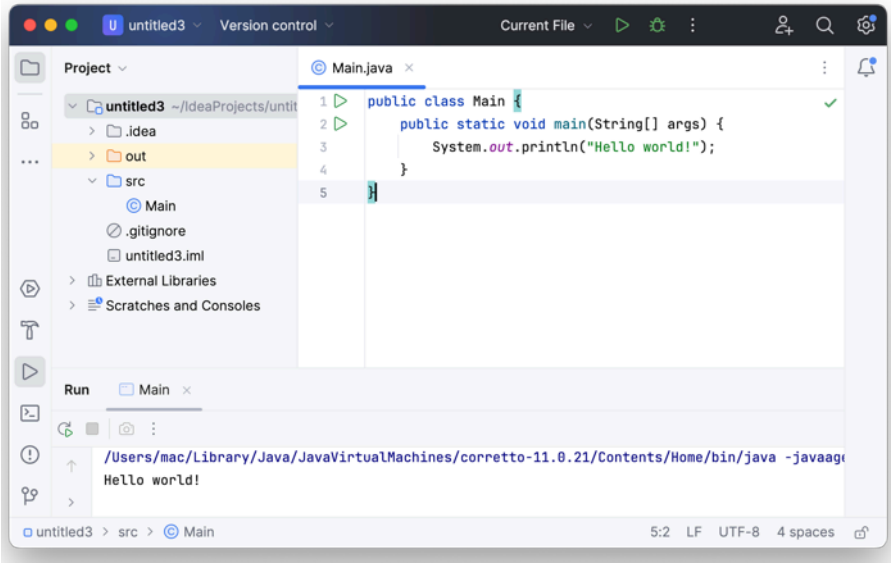


Figure 1.15. Main interface of the IntelliJ IDEA programming tool.

1.6 . STRUCTURE OF A JAVA FILE

A typical Java program has the following structure:

```
1   package com.example.laptrinh; //Khai báo gói
2
3   import java.util.Date;  //Câu lệnh Import
4
5   /**
6    * @author: Pham Xuan Lam            //Chú thích
7    */
8   public class Program {    //Khai báo tên Lớp
9
        1 usage
10      public static String data = "Lập trình Java";  //Khai báo biến thành viên
11
12      public static void main(String[] args) {     //Hàm Main
13          Program program = new Program();
14          program.printMessage(data);
15      }
16
        1 usage
17      public void printMessage(String mesage) {    //Các phương thức
18          Date date = new Date();
19          System.out.println(mesage + ": " + date);
20      }
21  }
```

Figure 1.16. Structure of a program in Java.

The basic structure of a Java program includes declaring many components:

- The keyword **package** is used alongside the package name to specify which package a Java class belongs to.
- **import** keyword is used to declare a library to be used in a Java file.
- The keyword **class** is used to declare a class in Java.
- The keyword **public** is an access modifier.
- **static** is a keyword, meaning that if we declare any method as static, it is called a static method. The main advantage of static methods is that it is not necessary to create an object to call the static method.
- **void** is the return type of a method, meaning the method does not

return any value.

- **main** represents the program's startup.
- **String[] args** are used for command-line parameters.
- **System.out.println()** is used as the command to print data to the Console.
- **Methods** are the methods within a Java file.

Java files are organized into a **project**, and this Java project runs on a **JDK (**Java Development Kit). To see which JDK version a project is based on, you can check the External Libraries folder.

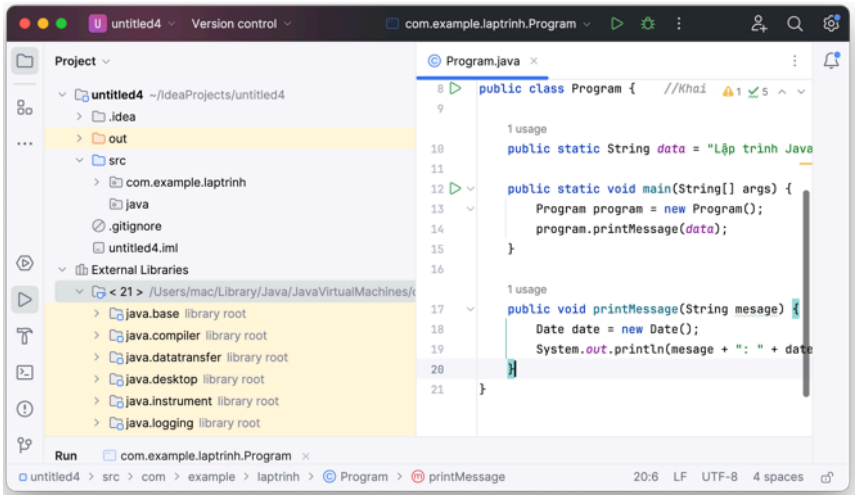


Figure 1-17. View the JDK version used in the Project

To change the JDK version, go to File -> Project Structures -> Project -> Project SDK:

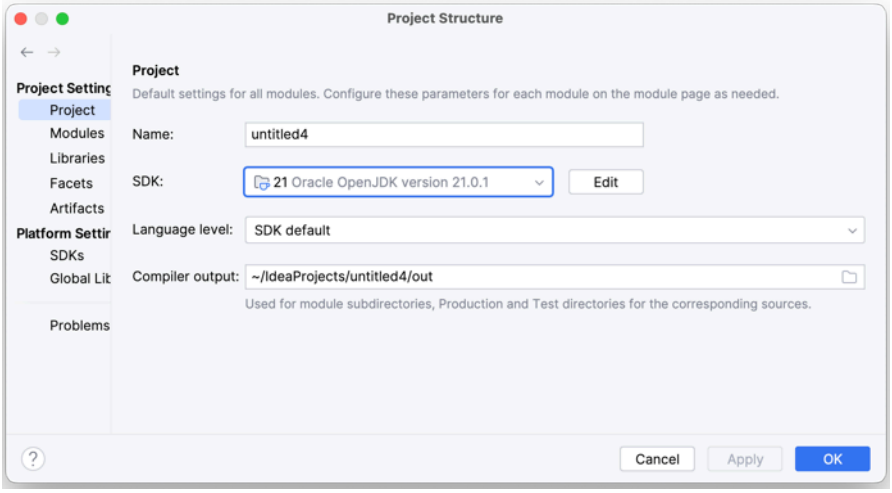


Figure 1 -18. Changing the JDK version used in the Project

Project Language Level option is used to determine which Java version features a programmer wants to use.

For example, if a programmer installs and uses JDK 11 but wants their source code to be compatible with Java 8, they would select Language Level 8. This means only features from Java 8 and earlier versions are allowed.

Java classes can be placed in packages. To create a package, right-click where you want to create the package (the src folder) and then select **New** -> **Package.**

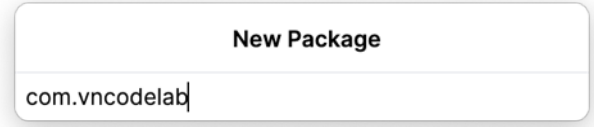


Figure 1 -19. Create a package

Java files that are not in any specific package are considered to be in **the default package**. Java packages create a directory structure for storing Java classes. A class declares its package name at the beginning of the file, for example, a **Helloworld class in the com.vncodelab** package**:**

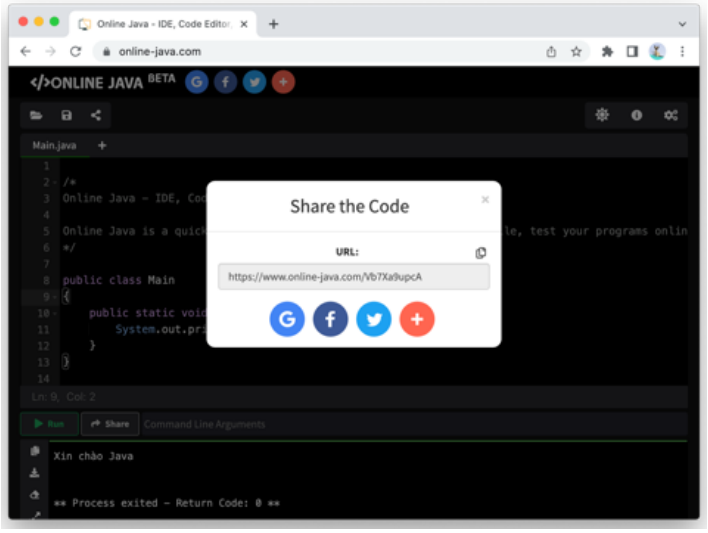
```
package com.vncodelab
class Helloworld {
}
```

PRACTICE

Lesson 1. Practicing using online tools

Access the page **https://www.online-java.com/** and a Java source code editor window will appear. The main method has a printf function to print a string to the screen. Change this string to "Hello Java" as shown in the image below, then click the Run button to execute:

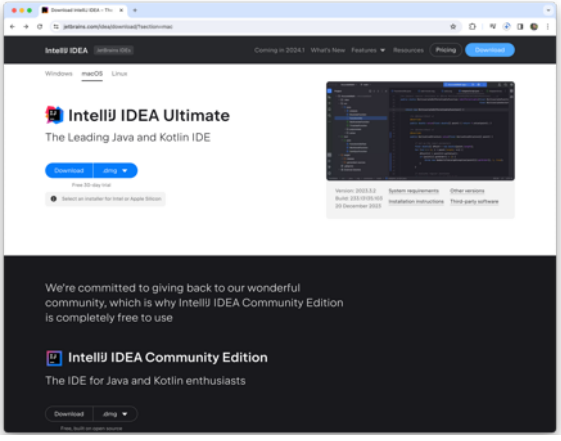
Click **"share"** to share the source code with others. The tool will then generate a link to send it to them.



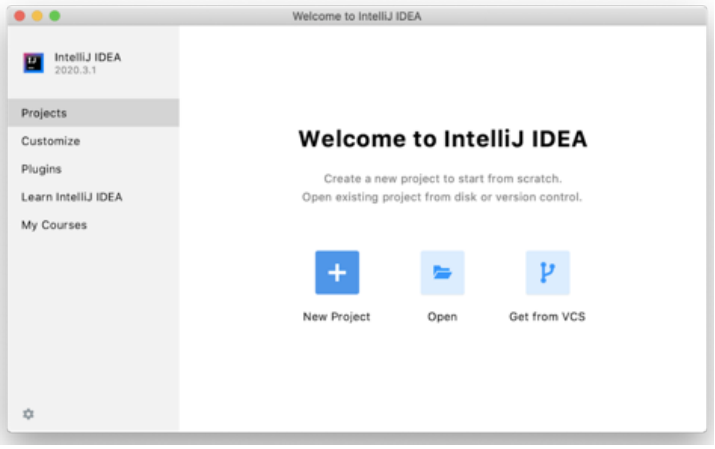
Lesson 2. Creating a Project using IntelliJ IDEA

The following section will guide you on how to download **IntelliJ IDEA** and create a Java **project** using this tool. First, you need to download **IntelliJ IDEA** from: **https://www.jetbrains.com/idea/download.**

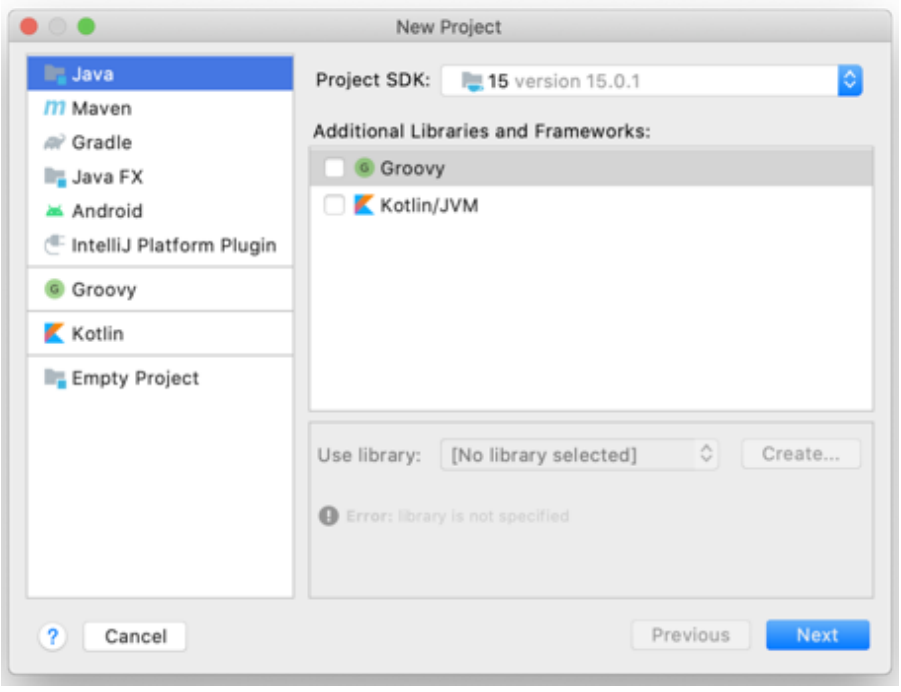
Please note that JetBrains currently offers two versions: a free version and a paid version. However, the paid version can be used for a 30-day or one-year trial (for student users with email accounts using the .edu domain).



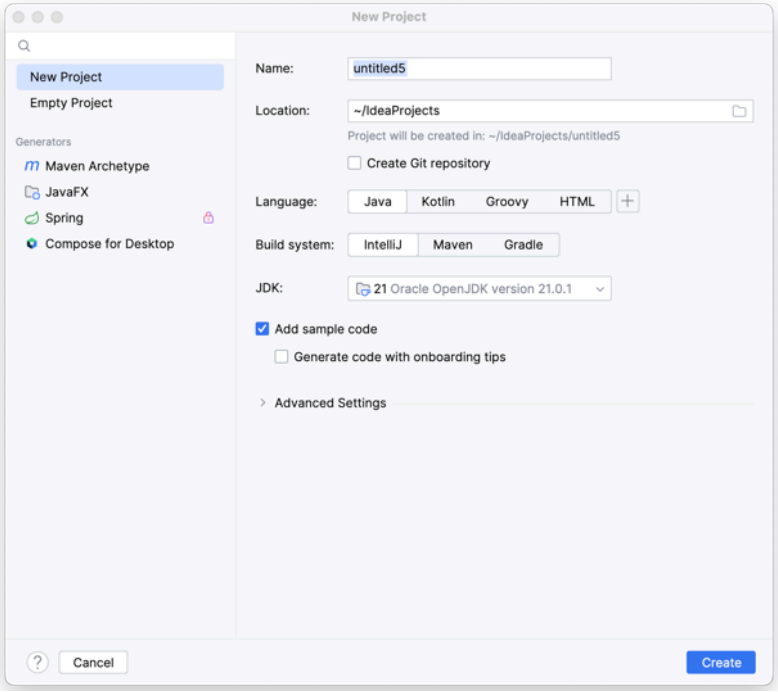
Project creation interface will appear. Note that the interfaces often change depending on the version used. The image below shows the free version of IntelliJ IDEA 2020.3.1.



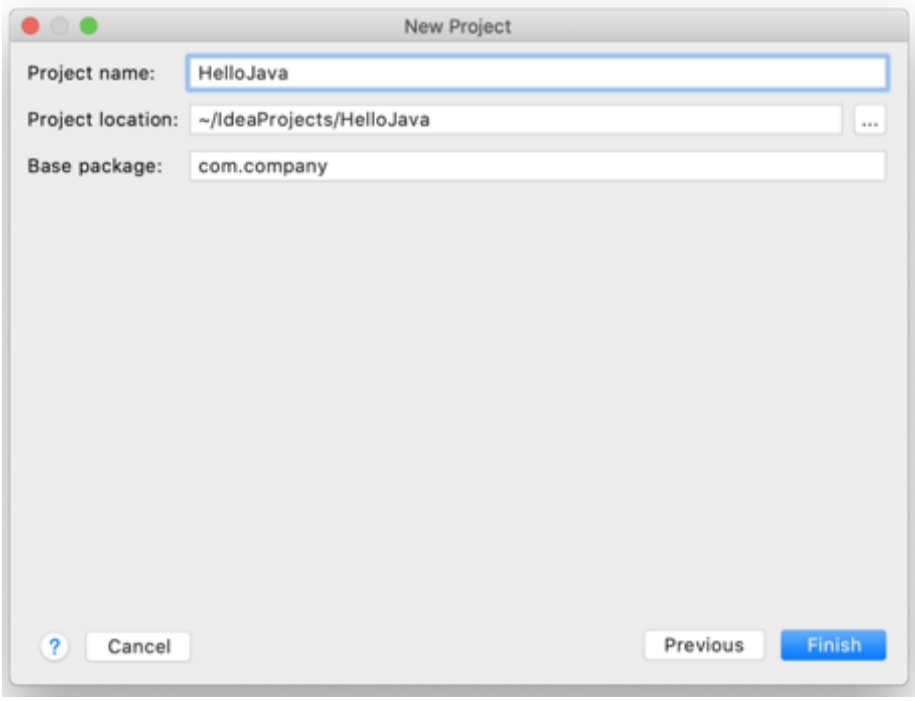
Select **"New Project"** to create a new project. The next step is to select **the SDK** (Software Development Kit); the image below shows the JDK that was previously installed.



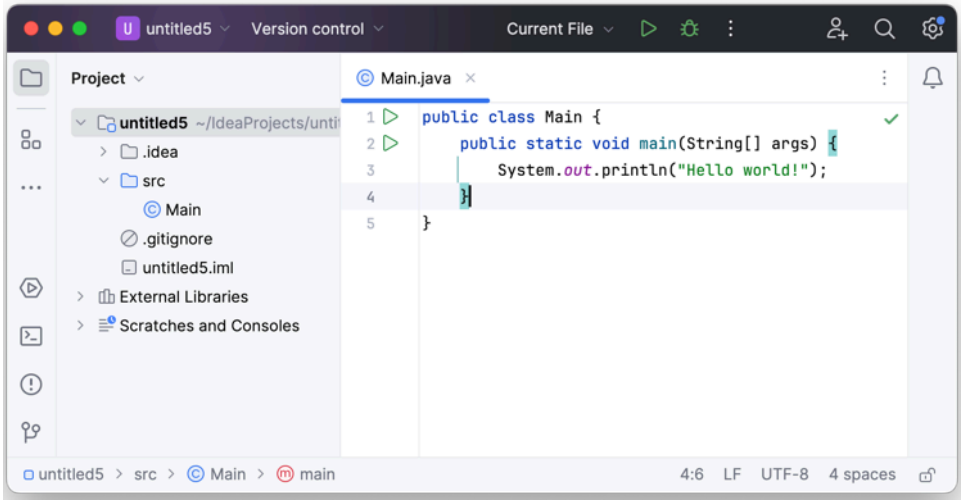
In the next step, select **Create project from template** to create a project from a pre-existing template:



The next step is **to name** the project (for small projects, the project name is usually the name of the application).



The programming interface screen will appear as follows:

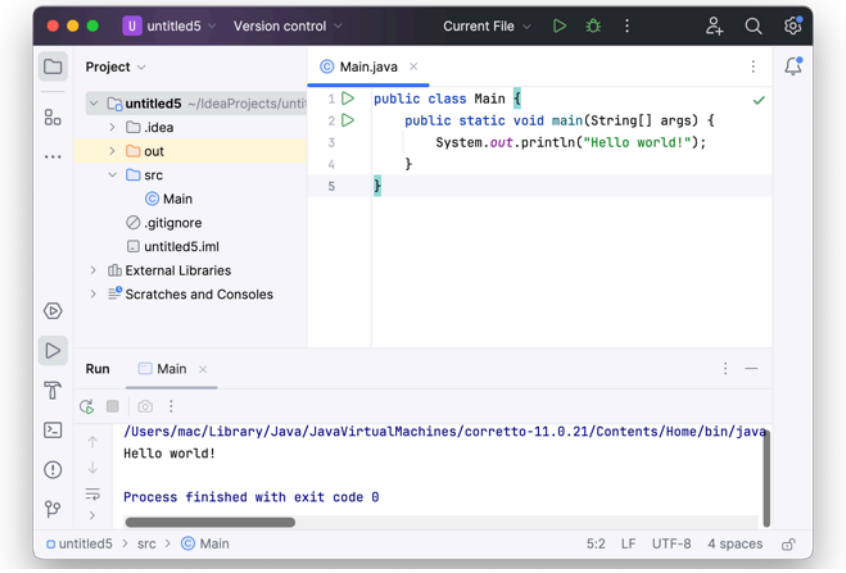


1. Modify the main() method to print "Hello Java":

```
public static void main(String[] args) {
System.out.println("Hello Java");
```

```
}
```

2. Run the test and check the printed output on the Console screen:



QUIZ

Which statement about the Java language is incorrect?	
The Java language is case-sensitive.	
Java is an object-oriented programming language.	
Semicolons are used to end a statement in Java.	
Programs written in Java can only run on the Windows operating system.	

To run a Java program, which of the following software do you NEED to install?	
IntelliJ IDEA	
Eclipse	
JDK	
JRE	

What components are not included in the CPU structure?	
ALU	
RAM	

CU	
Cache	

What components are not included in the Java Virtual Machine architecture?	
Classloader	
JVM memory	
Execution engine	
JDK	

What is a Java Virtual Machine?	
An engine provides a runtime environment for running Java code or applications	
It converts Java bytecode into a language that computers understand	
As part of JRE	
All options are correct	

What is the standard syntax for the main function?	
static void main(String args)	
public void main(String[] args)	
public static void main (String[] args)	
static void main(String ... args)	

To use a library package, use the command	
opera	
paper	
permission	
import	

What is Java LTS?	
It is a long-term supported version of Java	
This is the latest version of Java	
This is a test version of Java	
It is the free version of Java	

To compile a Java file, use the command:	
Java	
compile	
build	
javac	

Java is a programming language:	
Interpreted	
Compiled	
Compiled and Interpreted	
Procedural	

PRACTICE EXERCISES

Write a program that displays your personal information:

- Full name
- Student ID number
- Class name

	OJ Java

REFERENCES

[1] Core Java: Fundamentals (2021) , Cay Horstmann (Oracle Press Java).

[2] Head First Java: A Brain-Friendly Guide (2022), Kathy Sierra, O'Reilly Media.

[3] Java OOP Done Right: Create object oriented code you can be proud of with modern Java Paperback (2019), Mr Alan Mellor, Mellor Books.

[4] Murach's Java Programming (5th Edition) (2017), Joe Murach, Mike Murach & Associates.

[5]. Java for Absolute Beginners Learn to Program the Fundamentals the Java 9+ Way.

[6]. Modern Java Recipes: Simple Solutions to Difficult Problems in Java 8 and 9 (2017), by Ken Kousen, O'Reilly Media.

[7] Effective Java (2018), Joshua Bloch, Addison-Wesley Professional.

[8] Java Threads: Understanding and Mastering Concurrent Programming (2004), Scott Oaks, O'Reilly Media.

[9] Learning Java: An Introduction to Real-World Programming with Java (2020), Marc Loy, O'Reilly Media.

[10] Learn Java the Easy Way: A Hands-On Introduction to Programming (2017), Bryson Payne, No Starch Press.